CENTRALITY METRICS IN GRAPHBLAS

by

Karan Bhalla
Computer Science, College of Engineering

A Thesis Submitted to the Office of Undergraduate Research at
Texas A&M University
In partial fulfillment of the requirements for the designation of

UNDERGRADUATE RESEARCH SCHOLAR

May 2026

Approved by Faculty Research Advisor:

Dr. Timothy Davis
Computer Science and Engineering, College of Engineering
Texas A&M University

COMPLIANCE NOTICE

I, Karan Bhalla, certify that all research compliance requirements related to this Undergraduate Research Scholars thesis have been addressed with my Faculty Research Advisor prior to the collection of any data used in this document.

This project did not require approval from Research Compliance and Biosafety at Texas A&M University.

TABLE OF CONTENTS

Page

ABSTRACT

Centrality metrics measure the relative importance of nodes and edges within a graph, identifying those that are most influential. These metrics provide insight into network structure, helping to pinpoint key nodes that control information flow and play critical roles in connectivity. Traditional implementations of such algorithms are often iterative and don't scale well to large graphs. With GraphBLAS, graph algorithms can be expressed using linear algebra operations on matrices and vectors, allowing computations to be performed in parallel and with greater efficiency. This linear algebra based approach is particularly well-suited for handling sparse graphs. Sparse graphs are networks where edges are very few relative to the total number of possible connections and commonly appear in many real-world scenarios. This paper extends the suite of centrality metrics in SuiteSparse:GraphBLAS by implementing Katz and closeness centrality algorithms. Katz centrality measures a node's influence by considering all paths that connect it to other nodes, giving less weight to longer and indirect connections. Closeness centrality reflects how near a node is to all other nodes in the graph based on the shortest-path distances. We benchmark our implementations to evaluate their performance and compare them against existing centrality algorithms in SuiteSparse:GraphBLAS as well as other graph frameworks.

ACKNOWLEDGMENTS

Contributors

I would like to express my sincere gratitude to my faculty advisor, Dr. Timothy Davis, for his guidance and support throughout the course of this research. I would also like to thank my family for their constant encouragement.

Thanks also go to my friends, colleagues, and the department faculty and staff for making my time at Texas A&M University a truly memorable experience.

All other work conducted for the thesis was completed by the student independently.

Funding Sources

Undergraduate research was supported by Dr. Timothy Davis at Texas A&M University.

DEDICATION

To my mother, Charu, for her unconditional love and support.

1. INTRODUCTION

Graph-structured data arises naturally across a wide range of domains, including social networks, communication systems, biological networks, and large-scale information systems. In such scenarios, nodes represent entities while edges represent relationships between them. Extracting meaningful insights from such data relies on graph algorithms that quantify structural properties or identify influential nodes. Among these, centrality metrics play a crucial role by providing methods to assess the importance of individual nodes. This work extends the suite of centrality metrics in LAGraph, a rich library of high-level graph algorithms, by implementing two such metrics - Katz centrality and closeness centrality - and benchmarks them against existing implementations.

1.1. Graph Algorithms in Linear Algebra

Traditional graph algorithms are typically expressed using iterative, traversal-based techniques such as breadth-first search (BFS). While these approaches are conceptually intuitive, they can be difficult to scale efficiently to large graphs due to their sequential nature and irregular memory access patterns. As modern datasets grow in size and complexity, there is an increasing need for formulations that are computationally efficient.

An alternative and increasingly influential perspective expresses graph algorithms using linear algebra. By representing graphs as matrices and algorithmic operations as matrix and vector computations, this paradigm provides a mathematically concise and highly efficient framework for graph analysis. Many classical graph algorithms can be reformulated in this manner, and the resulting implementations directly inherit decades of research breakthroughs in numerical linear algebra without requiring algorithm-specific optimization work. Consequently, any improvements to the underlying linear algebra layer automatically benefit all graph algorithms built on top of it, making the approach not only efficient today but well-positioned to take advantage of future hardware and algorithmic advances.

In the linear algebraic formulation of graph algorithms, a graph with n nodes is commonly

represented by an adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ where the entry $\mathbf{A}_{ij}$ indicates the presence of an edge from node i to node j [1]. This representation enables graph algorithms to be expressed in terms of matrix and vector operations rather than explicit traversal of nodes and edges.

Many fundamental graph operations map naturally to matrix computations. For example, multiplying the adjacency matrix $\mathbf{A}$ by a vector $\mathbf{x}$ computes the sum of contributions from all neighboring nodes for each node. Similarly, powers of the adjacency matrix $\mathbf{A}^k$ encode walks of length k within the graph, making them useful for path-based analysis. These algebraic operations provide a compact and uniform way to describe neighborhood expansion and influence propagation.

A key advantage of this approach is the ability to exploit sparsity. Real-world graphs are typically sparse, meaning the number of edges is far smaller than the maximum possible connections. Sparse matrix representations store only nonzero entries, allowing both memory usage and computation time to scale with the number of edges.

Another idea central to this framework is the use of semirings, which generalize conventional arithmetic by redefining the addition and multiplication operations. By selecting different semiring operations, the same matrix computations can express different graph semantics, such as shortest paths, reachability, or weighted influence. This abstraction allows a small set of algebraic operators to support a wide range of graph algorithms.

1.2. Katz Centrality

Katz centrality is a measure of influence that takes into account both direct and indirect connections between nodes within a network. Unlike some centrality methods which only consider immediate neighbors, Katz centrality assigns importance to a node based on the number and quality of all paths that connect it to other nodes in the network: longer paths contribute less to a node's centrality than shorter ones. Katz centrality for node i is defined as:

$$x_i = \alpha \sum_{j=1}^{n} A_{ij} x_j + \beta \tag{1}$$

where x_i is the Katz centrality, α is the attenuation factor, and β is constant bias term.

1.3. Closeness Centrality

Closeness centrality measures how near a node is to all other nodes in a network, marking a node as important if others can reach it quickly. It is particularly valuable in applications where accessibility and fast information propagation are critical. For a graph with n nodes, the closeness centrality of node u is defined as the reciprocal of the average shortest-path distance to u over all $n-1$ reachable nodes:

$$C(u) = \frac{n-1}{\sum_{v \neq u} d(v,u)} \tag{2}$$

where $d(v,u)$ is the shortest-path distance from node v to node u, and $n-1$ is the number of nodes reachable from u. This formulation uses incoming shortest-path distances to u rather than outgoing distances from u to others. These distances are identical for undirected graphs. However, for directed graphs these distances differ and the choice of direction meaningfully changes which nodes are considered central.

Unlike Katz centrality, which aggregates influence through attenuated walks of all lengths, closeness centrality depends solely on shortest-path distances. Consequently, its computation reduces to solving a shortest-path problem from every node in the graph, making it significantly more expensive than Katz centrality at scale.

2. METHODS

2.1. SuiteSparse:GraphBLAS

The GraphBLAS standard defines graph algorithms in terms of sparse linear algebra operations. SuiteSparse:GraphBLAS is a high-performance implementation of this standard developed by Dr. Timothy Davis [2]. Rather than prescribing specific graph algorithms, the standard provides a set of primitives - such as matrix-vector multiplication, matrix-matrix multiplication, element-wise operations, and reductions - that can be composed to express a wide range of graph computations. This abstraction enables graph computations to inherit decades of optimization work from sparse linear algebra while maintaining algorithmic generality and readability.

2.1.1. Graph Representation

A central concept in GraphBLAS is the representation of graphs as matrices and vectors. The well-established duality between graphs and adjacency matrices provides the theoretical foundation for this approach: a graph $G = (V, E)$ with $|V| = n$ can be represented as an $n \times n$ matrix $\mathbf{A}$, where the entry $\mathbf{A_{ij}}$ encodes the presence (and optionally the weight) of an edge from vertex i to vertex j. Undirected graphs correspond to symmetric matrices, while directed graphs are represented by general matrices.

In most real-world settings, graphs are sparse. Although the adjacency matrix conceptually contains n^2 entries, only a small fraction correspond to actual edges. SuiteSparse:GraphBLAS therefore stores matrices using sparse formats such as compressed sparse column (CSC) and hypersparse variants, where only nonzero entries are explicitly recorded. Nonzero entries are recorded through three arrays: a column pointer array that marks the start of each column, a row index array that identifies the row of each stored entry, and a value array that holds the corresponding entry values. Entries equal to the additive identity (the implied zero) are omitted entirely. As a result, memory consumption scales with the number of edges m, rather than the number of possible vertex pairs.

For example, consider a graph with one million vertices and average degree 10. A dense representation would require storage for 10^{12} entries, whereas a sparse representation stores only the 10^7 actual edges and indexing metadata. Matrix–vector multiplication then operates only on stored entries, allowing traversal-based algorithms such as breadth-first search or power iteration methods for centrality to scale efficiently.

In addition to sparse and hypersparse formats, SuiteSparse:GraphBLAS introduced bitmap matrix representation in v4.0.3 release [1]. The bitmap format occupies an intermediate position between sparse and dense storage. Instead of maintaining explicit index arrays for each nonzero entry, a bitmap matrix allocates space for all possible entries but accompanies the value array with a Boolean bitmap that indicates which positions are logically present. Subsequent computation consults this bitmap to determine whether a given entry participates in an operation.

This representation becomes advantageous when matrices are neither extremely sparse nor fully dense. In such scenarios, sparse formats incur overhead from storing and traversing index structures, while dense formats perform unnecessary work on zero entries. The bitmap format reduces index overhead while still avoiding computation on absent values, providing a balanced trade-off between memory usage and arithmetic efficiency.

Vectors follow analogous principles. Sparse vectors store only indices with present values, which is ideal for frontier-style computations where activity is localized. Bitmap vectors, on the other hand, provide efficient access patterns when support becomes widespread. Importantly, matrices and vectors in GraphBLAS need not be square. Rectangular matrices naturally represent bipartite graphs or incidence structures, further reinforcing the flexibility of the representation.

A key strength of SuiteSparse:GraphBLAS lies in its abstraction of these storage decisions. Users manipulate matrices and vectors through a consistent algebraic interface, without managing internal formats directly. The implementation may dynamically select or convert between sparse, hypersparse, bitmap, and dense representations based on structural characteristics and workload. This adaptability is essential for centrality metrics and other graph algorithms whose computational patterns evolve over time, ensuring that performance remains efficient across varying degrees of

sparsity.

2.1.2. Semirings

2.1.2.1. Definition and Properties

A semiring is an algebraic structure $(R, \oplus, \otimes)$ consisting of a set R equipped with two binary operators: an additive operator $\oplus$ and a multiplicative operator $\otimes$. The additive operator forms a commutative monoid, while the multiplicative operator forms a monoid. The additive operator $\oplus$ satisfies:

- **Closure:** $x \oplus y \in R$ for all $x, y \in R$
- **Associativity:** $(x \oplus y) \oplus z = x \oplus (y \oplus z)$
- **Commutativity:** $x \oplus y = y \oplus x$
- **Identity:** there exists $0 \in R$ such that $x \oplus 0 = 0 \oplus x = x$

Thus, $(R, \oplus)$ is a commutative monoid with identity element 0. The multiplicative operator $\otimes$ satisfies:

- **Closure:** $x \otimes y \in R$ for all $x, y \in R$
- **Associativity:** $(x \otimes y) \otimes z = x \otimes (y \otimes z)$
- **Identity:** there exists $1 \in R$ such that $x \otimes 1 = 1 \otimes x = x$

Hence, $(R, \otimes)$ is a monoid with identity element 1.

In a classical semiring, $\otimes$ distributes over $\oplus$ from both sides, and the additive identity 0 annihilates under $\otimes$, meaning $0 \otimes x = x \otimes 0 = 0$ for all $x \in R$. The familiar arithmetic structure $(\mathbb{R}, +, \times)$ is a canonical example, though many other algebraic systems, including Boolean logic or min-plus arithmetic also satisfy these properties.

The importance of semirings lies in their role in redefining matrix multiplication. Under a semiring, the product of matrices A and B is given by

$$C_{ij} = \bigoplus_{k} A_{ik} \otimes B_{kj} \tag{3}$$

which preserves the multiply-accumulate structure of classical matrix multiplication while replacing standard addition and multiplication. This abstraction allows matrix operations to encode a wide range of graph computations within a unified algebraic framework.

2.1.2.2. GraphBLAS Semirings

While GraphBLAS semirings are inspired by the classical algebraic definition, they deliberately relax certain constraints to increase flexibility and performance. In GraphBLAS, a semiring consists of:

- An additive monoid $(D_{\text{out}}, \oplus)$
- A multiplicative operator $\otimes : D_{\text{in}_1} \times D_{\text{in}_2} \to D_{\text{out}}$

Unlike classical semirings, the input domains of $\otimes$ need not coincide with the output domain. The multiplicative operator may take arguments from two possibly distinct domains and produce a value in a third, allowing mixed-type computations — for example, multiplying integer adjacency values with floating-point weights and accumulating the result in double precision.

Furthermore, GraphBLAS does not strictly require full distributivity in the formal algebraic sense. While distributive behavior is preserved in practical implementations of matrix multiplication, the API permits greater flexibility in operator definitions than the strict mathematical definition would allow. This relaxation broadens the class of admissible operators while preserving efficient kernel implementations.

The annihilation property of the additive identity remains essential. When $0 \otimes x = x \otimes 0 = 0$, GraphBLAS can exploit sparsity safely: missing entries are treated as implicit zeros, and multiplications involving absent values do not contribute to the accumulation. Without annihilation, sparse kernels would require explicit storage of zero entries, defeating the purpose of sparse representation.

GraphBLAS semirings therefore serve a dual purpose. Algebraically, they define the semantics of matrix operations. Computationally, they enable efficient sparse implementations by ensuring compatibility with the implied-zero model.

For example, the `PLUS_TIMES` semiring corresponds to classical arithmetic multiplication and is used in power iteration methods such as Katz centrality. The `OR_AND` semiring captures reachability in Boolean graph traversal. The `MIN_PLUS` semiring reinterprets multiplication as addition and addition as minimum, enabling shortest-path propagation. In each case, the underlying matrix multiplication kernel is identical and only the semiring changes.

2.1.2.3. Masks

In many graph algorithms, it is necessary to restrict where results are written. For example, an algorithm may need to update only unvisited vertices during a traversal, or modify only a specific subset of nodes during a centrality computation. GraphBLAS handles this through masks which are opaque objects that act as filters on assignment. A mask has the same dimensions as the output matrix or vector, and any index not selected by the mask leaves the corresponding output entry unchanged. This allows conditional updates to be expressed algebraically rather than through explicit branching logic.

Masks come in two forms. A value-based mask permits assignment at a given index only if the stored value at that index evaluates to true. A structural mask depends solely on the presence of an entry, regardless of its numerical value, making it more efficient in practice since it requires only sparsity pattern information rather than value comparisons.

To illustrate the use of masks, consider the breadth-first search algorithm. At each step, one maintains a vector $\mathbf{v}$ marking already-visited vertices and a frontier vector $\mathbf{f}_k$ representing the active front at level k. The next frontier is computed as $\mathbf{f}_{k+1} = \mathbf{A}\mathbf{f}_k$ but this product may include vertices that have already been visited. Rather than filtering these out with an explicit loop, applying the complement of $\mathbf{v}$ as a structural mask on the output ensures that only previously unvisited vertices are written into $\mathbf{f}_{k+1}$. Visited vertices are automatically excluded by the mask without any conditional logic in the algorithm itself.

This mechanism generalizes naturally to a wide range of graph algorithms. In centrality computations, masks can restrict score updates to a specific subset of vertices. In shortest-path algorithms, they can prevent overwriting already-settled distances.

2.1.3. Descriptors

Descriptors are lightweight flags that modify how inputs, masks, or outputs are interpreted during an operation. They do not alter the underlying data structures; instead, they adjust the semantics of execution.

Common descriptor options include transposing an input matrix, complementing a mask, treating a mask as structural, or specifying that the output should be replaced rather than accumulated into. For example, in directed graphs, it is often necessary to propagate information along incoming edges. Rather than explicitly forming $\mathbf{A}^T$, a descriptor can instruct the multiplication kernel to treat the input as transposed. Similarly, the complement descriptor reverses mask behavior: assignments occur where the mask is absent rather than present. The replace descriptor ensures that the output is cleared before writing new results, preventing unintended accumulation across iterations.

Descriptors provide fine-grained operational control while preserving the high-level algebraic formulation. Together with sparse representations, semirings, and masks, they form the core abstraction that enables SuiteSparse:GraphBLAS to express complex graph algorithms concisely and execute them efficiently at scale.

2.2. Katz Centrality

2.2.1. Overview

Katz centrality is a measure of node influence that accounts for both direct and indirect connections in a network. It was introduced by Leo Katz in 1953 [3], who recognized that a node's importance should reflect not just its immediate neighbors, but the entire web of relationships it participates in.

More formally, let $A \in \mathbb{R}^{n \times n}$ denote the adjacency matrix of a graph with n vertices. The

Katz centrality vector $x \in \mathbb{R}^n$ is defined as the solution to

$$x = \alpha A x + \beta \mathbf{1} \tag{4}$$

where $\alpha \in \mathbb{R}$ is the attenuation factor, $\beta \in \mathbb{R}$ is a constant bias term, and $\mathbf{1}$ is the all-ones vector. The attenuation factor α controls how much weight is given to longer paths: a walk of length k contributes a factor of α^k to the centrality score, so longer paths matter progressively less. The bias term β ensures that every vertex receives a baseline score regardless of its connectivity, preventing isolated nodes from receiving a score of zero.

When the inverse exists, **Equation 4** has a closed-form solution:

$$x = (I - \alpha A)^{-1} \beta \mathbf{1} \tag{5}$$

For this inverse to exist and for the centrality scores to remain finite, α must satisfy

$$\alpha < \frac{1}{\lambda_{\text{max}}} \tag{6}$$

where λ_{max} is the largest eigenvalue of A. Intuitively, if α is too large, the contributions from very long walks grow without bound rather than diminishing, causing the centrality scores to diverge. Keeping α below $1/\lambda_{\text{max}}$ guarantees that longer walks contribute less and less, so the sum remains finite.

In practice, Katz centrality is computed iteratively using the recurrence

$$x^{(k+1)} = \alpha A x^{(k)} + \beta \mathbf{1} \tag{7}$$

starting from an initial vector $x^{(0)}$. At each step, the current score vector is multiplied by A and scaled by α, propagating influence from neighbors, and then shifted by $\beta \mathbf{1}$ to maintain the baseline. This is repeated until the solution stabilizes, which is determined by monitoring the L1 norm of successive iterations $\|x^{(k+1)} - x^{(k)}\|_1$. Iteration terminates when the L1 norm falls below a

prescribed tolerance ϵ.

2.2.2. Implementation in SuiteSparse

Algorithm 1 presents the implementation of `LAGr_KatzCentrality`. The implementation supports both directed and undirected graphs. For undirected graphs, or graphs with symmetric structure, the adjacency matrix A is used directly. For directed graphs, the transposed matrix A^T is used, cached as `G->AT`, so that centrality scores reflect incoming walks rather than outgoing ones.

Algorithm 1: *Katz Centrality*

Require: $A \in \mathbb{R}^{n \times n}$ (adjacency matrix; A^T used for directed graphs)
Require: $\alpha \in (0, 1/\lambda_{\text{max}})$, $\beta > 0$
Require: tolerance $\epsilon > 0$, max iterations K, `use_weights`, `normalize`
Ensure: $x \in \mathbb{R}^n$ (Katz centrality vector)

```
x ← 0, x_prev ← 0
b ← β · 1
for k = 0, 1, ..., K − 1 do
    swap x ↔ x_prev
    x ← A^T x_prev                ▷ mxv over PLUS_TIMES_FP64 or PLUS_SECOND_FP64
    x ← α · x
    x ← x + b
    rdiff ← ||x − x_prev||_1
    if rdiff < ε then
        break                     ▷ Convergence achieved
    end if
end for
if normalize then
    x ← x / ||x||_2
end if
```

The semiring used in the matrix-vector multiplication depends on the `use_weights` flag. For weighted graphs, the standard `PLUS_TIMES_FP64` semiring is used, accumulating weighted walk contributions. For unweighted graphs, `PLUS_SECOND_FP64` is used instead, which replaces the multiplicative operator with the `SECOND` operator, effectively treating all edges as having unit weight and avoiding unnecessary multiplications and edge weight memory accesses. When edge weights are used, the implementation additionally validates that all weights are nonnegative,

since negative weights would violate the convergence condition $\alpha < 1/\lambda_{\max}$.

Each iteration of the algorithm performs three core GraphBLAS operations: a matrix-vector multiply (`GrB_mxv`), a scalar scaling via `GrB_apply`, and an element-wise addition of the bias vector b via `GrB_eWiseAdd`. Convergence is checked by computing the L1 norm of the absolute difference between successive iterates using `GrB_apply` with `GrB_ABS`, followed by `GrB_reduce`. An optional normalization step divides the result by its L2 norm, computed as $\sqrt{x^T x}$ via element-wise multiplication and reduction.

2.2.3. Relationship to PageRank

PageRank is a widely used centrality measure that assigns scores to nodes based on the importance of their incoming neighbors. The `LAGr_PageRankGAP` function implements PageRank in LAGraph as an iterative, power-method style algorithm using SuiteSparse:GraphBLAS primitives, following the algorithmic specification of the GAP Benchmark Suite [4].

Algorithm 2 presents the pseudocode for `LAGr_PageRankGAP`.

Algorithm 2: *GAP PageRank*

Require: $A \in \mathbb{R}^{n \times n}$ (adjacency matrix; A^T used for directed graphs)
Require: damping factor d, tolerance $\epsilon > 0$, max iterations K
Require: $d_{\text{out}} \in \mathbb{R}^n$ (cached out-degree vector)
Ensure: $r \in \mathbb{R}^n$ (PageRank vector)

```
r ← (1/n) · 1
d̂ ← max(d_out/d, (1/d) · 1)          ▷ Pre-scaled degree, clamped to 1/d
teleport ← (1 − d)/n
for k = 0, 1, ..., K − 1 do
    swap t ↔ r                        ▷ t holds old scores
    w ← t ⊘ d̂                         ▷ Element-wise division
    r ← teleport · 1
    r ← r + A^T · w                   ▷ mxv over PLUS_SECOND_FP32
    rdiff ← ||r − t||_1
    if rdiff ≤ ε then
        break
    end if
end for
```

Each iteration of `LAGr_PageRankGAP` performs a series of core GraphBLAS operations

that closely mirror those in `LAGr_KatzCentrality`. First, the previous iteration's scores are cached by swapping vectors. Next, the vector of scores is normalized by the pre-scaled out-degree and combined with a teleportation vector. This is followed by a matrix-vector multiplication with the adjacency matrix, accumulating contributions from incoming neighbors. Finally, convergence is assessed by computing the L1 norm of the difference between successive iterates. These steps are analogous to the iterative scaling, bias addition, and convergence check in Katz centrality, highlighting the structural similarity between the two algorithms while differing only in the details of the update and normalization operations.

2.3. Closeness Centrality

2.3.1. Overview

Closeness centrality was first formalized by Alex Bavelas in 1950 in the study of communication patterns in task-oriented groups [5]. This work focuses on the formulation introduced in **Equation 2**, which assumes a connected graph where all nodes are reachable from every source. It is worth noting, however, that real-world graphs are often disconnected, and the basic formula breaks down when some nodes are unreachable since the sum of distances becomes undefined.

Wasserman and Faust [6] proposed a normalization that addresses this by scaling the score by the fraction of reachable nodes:

$$C_{\text{WF}}(u) = \frac{R(u)}{n-1} \cdot \frac{R(u)}{\sum_{v \neq u} d(v,u)} \tag{8}$$

where $R(u)$ is the number of nodes that can reach u and n is the total number of nodes in the graph. This ensures that a node reachable from only a small fraction of the graph cannot achieve an artificially high score. A full implementation of this variant is left as future work.

The computational challenge of closeness centrality stems from the fact that shortest-path distances must be computed from every node in the graph. For large graphs, this makes full evaluation expensive, and the choice of shortest-path algorithm has a significant impact on practical performance.

2.3.2. *Implementation in SuiteSparse:GraphBLAS*

`LAGr_ClosenessCentrality` is designed as a meta-algorithm: rather than prescribing a single shortest-path subroutine, it accepts an algorithm selector (`cc_algo_t`) that determines whether breadth-first search (BFS), delta-stepping single-source shortest path (SSSP), Bellman-Ford, or Floyd-Warshall is used internally. This design gives the user full control over which shortest-path algorithm is used, allowing them to best match the algorithm to their use case. As a secondary benefit, it also allows the algorithm to be tailored to the structure of the input graph: BFS for unweighted graphs, delta-stepping for non-negative weighted graphs, Bellman-Ford for graphs with negative weights, and Floyd-Warshall for small dense graphs requiring all-pairs distances.

For the Floyd-Warshall path, the implementation calls `LAGraph_FW`, a utility function contributed alongside this work that computes all-pairs shortest paths and returns the full distance matrix $D \in \mathbb{R}^{n \times n}$. To support this computation on dense intermediate matrices, a bitmap matrix representation was introduced into the Floyd-Warshall implementation, enabling the algorithm to operate on the all-pairs distance matrix as it fills in during computation.

For directed graphs, the implementation constructs a temporary graph `G_AT` wrapping the transposed adjacency matrix A^T, so that shortest-path traversals from a source node v in `G_AT` compute incoming distances to v in the original graph, correctly reflecting the definition of closeness centrality under directed edge semantics.

Algorithm 3 presents the pseudocode for `LAGr_ClosenessCentrality`. For the Floyd-Warshall path, `LAGraph_FW` is called once to produce the full distance matrix D. Self-loops are then removed via `GrB_select` with `GrB_OFFDIAG` to exclude self-distances from the sums. Row sums are computed using `GrB_reduce`, and reachable node counts are obtained by multiplying D against a dense all-zeros vector using the `plus_one` semiring, which counts the number of present entries in each row rather than summing their values. The centrality vector is then computed in a single element-wise division via `GrB_eWiseMult` with `GrB_DIV_FP64`.

For the per-node paths (BFS, SSSP, Bellman-Ford), the algorithm iterates over each source

Algorithm 3: *Closeness Centrality*

Require: G: input graph with adjacency matrix $A \in \mathbb{R}^{n \times n}$
Require: `sources`: set of source nodes to score; `NULL` or empty $\Rightarrow$ all nodes
Require: `algorithm` $\in$ {BFS, SSSP, Bellman-Ford, Floyd-Warshall}
Require: `use_weights`: whether to use edge weights
Ensure: $C \in \mathbb{R}^n$ (closeness centrality vector)

```
C ← 0
if algorithm = Floyd-Warshall then
    D ← LAGraph_FW(A)                                   ▷ All-pairs shortest-path matrix
    Remove diagonal entries from D                      ▷ Exclude self-distances
    dist_sums[v] ← Σ_u D[v,u] for all v                 ▷ GrB_reduce over rows
    R[v] ← nnz(row v of D) for all v                    ▷ Count reachable nodes via mxv with plus_one
    C ← R ⊘ dist_sums                                   ▷ Element-wise division
    return C
end if
for each source node v in sources do
    if algorithm = BFS then
        ℓ ← LAGr_BreadthFirstSearch(G_trav, v)
        Remove ℓ[v]                                     ▷ Exclude self
        R(v) ← nnz(ℓ);  dist_sum ← Σ_u ℓ[u]
    else if algorithm = SSSP then
        d ← LAGr_SingleSourceShortestPath(G_trav, v)
        Prune entries where d[u] = ∞; remove d[v]
        R(v) ← nnz(d);  dist_sum ← Σ_u d[u]
    else if algorithm = Bellman-Ford then
        d ← LAGraph_BF_basic(A, v)
        if negative-weight cycle detected then
            C[v] ← NaN; continue
        end if
        Remove d[v]
        R(v) ← nnz(d);  dist_sum ← Σ_u d[u]
    end if
    if R(v) > 0 and dist_sum > 0 then
        C[v] ← R(v) / dist_sum
    end if
end for
return C
```

node, running the selected shortest-path subroutine on `G_trav`, the graph over the incoming adjacency. For BFS, `LAGr_BreadthFirstSearch` returns a level vector ℓ where $\ell[u]$ is the hop-distance from v to u; the self-entry is removed and the remaining entries are summed and counted. For SSSP, `LAGr_SingleSourceShortestPath` returns weighted distances, with unreach-

able nodes pruned via `GrB_select` before reduction. For Bellman-Ford, `LAGraph_BF_basic` is used, with negative-weight cycle detection handled by checking for a `GrB_NO_VALUE` return code; affected nodes receive a `NaN` score and are skipped. In all per-node cases, the closeness score is set only when both $R(v) > 0$ and *dist_sum* > 0, ensuring isolated or unreachable nodes retain a score of zero.

3. RESULTS

3.1. Contributions to LAGraph

This work implements two new centrality algorithms along with a supporting utility method in the experimental branch of the LAGraph repository. The `LAGr_KatzCentrality` function provides a scalable, iterative implementation of Katz centrality using matrix–vector multiplications over the appropriate semiring. Similarly, `LAGr_ClosenessCentrality` supports closeness centrality computations for all nodes or a selected subset, and allows the user to select the shortest-path algorithm used internally. This includes an all-pairs shortest-path computation via `LAGraph_FW`, a utility function implementing the Floyd-Warshall algorithm contributed alongside the centrality algorithms. The full code for the centrality algorithm implementations is available in the appendix and the LAGraph repository. Comprehensive test files were created to ensure correctness of the two centrality algorithms, and demo files illustrate usage patterns as well as benchmark runtime performance and efficiency.

3.2. Benchmarking

All benchmarking was conducted on a machine with a 12th Gen Intel(R) Core(TM) i7-1255U CPU @ 1.7–4.7 GHz with 12 threads. The GraphBLAS implementations of the centrality algorithms were compared against their counterparts in NetworkX, a popular Python library for network and graph analysis.

3.2.1. Katz Centrality

Table 1 summarizes the properties of the five graphs used to benchmark Katz centrality. All graphs were sourced from the SNAP collection [7]. The first two graphs are undirected, while the remaining three are directed. Collectively, these graphs cover a broad range of sizes and connectivity patterns.

Table 2 reports the runtime of GraphBLAS Katz centrality compared to NetworkX across

Table 1: *Properties of benchmarking graphs for Katz centrality*

Graph	Nodes	Edges	$\lambda_{\max}$
com-Youtube	1,134,890	5,975,248	210.395
com-LiveJournal	3,997,962	69,362,378	448.783
email-Enron	36,692	367,662	118.418
web-Stanford	281,903	2,312,497	48.376
wiki-Talk	2,394,385	5,021,410	228.940

* $\lambda_{\max}$ denotes the largest eigenvalue of the adjacency matrix

all benchmark graphs. For all experiments, the attenuation factor (α) was set to $\frac{0.85}{\lambda_{\max}}$ to ensure convergence, the bias term (β) was set to 1.0, and the maximum number of iterations was fixed at 1000. Note that NetworkX multiplies the tolerance by the number of nodes in the graph, which can be problematic for large graphs. To maintain comparable convergence criteria, the GraphBLAS tolerance was set to $1e{-}6$ and the tolerance passed to the NetworkX was divided by the number of nodes in the graph.

GraphBLAS achieves substantial speedups on all graphs where NetworkX completes, ranging from $213\times$ on `web-Stanford` to over $1,000\times$ on `email-Enron`. NetworkX fails to complete on `com-LiveJournal`, highlighting the scalability limitations of traditional implementations for larger graphs.

Table 2: *Runtime comparison of Katz centrality between GraphBLAS and NetworkX*

Graph	GraphBLAS time	NetworkX time	Speedup
com-Youtube	1.288 s	711.083 s	552.204
com-LiveJournal	14.007 s	-	-
email-Enron	0.026 s	26.426 s	1014.667
web-Stanford	1.467 s	312.270 s	212.929
wiki-Talk	3.684 s	1033.524 s	280.540

* Speedup is ratio of NetworkX time to GraphBLAS time

To further demonstrate the performance of our algorithm, we compare it against the GAP Benchmark Suite implementation of PageRank (GAP PageRank). As discussed previously, Katz centrality and PageRank share a similar iterative structure - both are power-iteration style algorithms - making PageRank a natural point of comparison. The GAP Benchmark Suite is a widely used collection of high-performance graph kernels designed to provide standardized, optimized

baselines for graph processing across diverse architectures [4]. Prior work has shown that SuiteSparse:GraphBLAS achieves strong performance on this benchmark for PageRank, where it attains upwards of 85% efficiency on many real-world and synthetic graphs [8].

Table 3 compares GraphBLAS Katz centrality against GAP PageRank in terms of iteration count, runtime, and per-iteration cost. Katz centrality consistently requires more iterations to converge than PageRank across all graphs which is expected due to its stricter convergence criterion. However, the per-iteration cost of Katz is comparable to PageRank, differing by about $1.2\times$ to $2\times$. This gap can be primarily attributed to the use of `FP_64` semirings and datatypes in Katz centrality, compared to `FP_32` in GAP PageRank, resulting in increased memory traffic per iteration.

Overall, these results confirm that Katz centrality exhibits comparable per-iteration computational characteristics to PageRank, and that its higher total runtime is driven primarily by a stricter convergence criterion rather than inefficiencies in the underlying implementation. Given the demonstrated efficiency of GraphBLAS on the GAP benchmark, we can reasonably infer that the performance of our Katz centrality implementation is likewise competitive.

Table 3: *Performance comparison between GraphBLAS Katz centrality and GAP benchmark PageRank (PR)*

Graph	Katz iters.	PR iters.	PR time	Katz time/iter	PR time/iter
com-Youtube	150	37	0.168 s	0.00859 s	0.00454 s
com-LiveJournal	129	24	1.434 s	0.10858 s	0.05975 s
email-Enron	134	36	0.006 s	0.00019 s	0.00017 s
web-Stanford	131	36	0.288 s	0.01120 s	0.00801 s
wiki-Talk	147	11	0.148 s	0.02506 s	0.01345 s

* GraphBLAS Katz centrality runtime reported in **Table 2**
† Iters. is number of iterations to convergence

3.2.2. Closeness Centrality

Table 4 summarizes the properties of the four graphs used to benchmark closeness centrality. The first graph, `bcsstk13`, is sourced from the `LAGraph/data` repository, while the remaining three are drawn from the SNAP collection [7]. The first two graphs are undirected and the last two are directed.

In all experiments, graphs are treated as unweighted and breadth first search (BFS) is

Table 4: *Properties of benchmarking graphs for closeness centrality*

Graph	Nodes	Edges
bcsstk13	2003	42,943
ca-HepPh	12,008	237,010
p2p-Gnutella25	22,687	54,705
email-Enron	36,692	367,662

used as the shortest-path subroutine, as NetworkX's closeness centrality implementation also defaults to BFS for unweighted graphs, ensuring a fair comparison. **Table 5** reports the runtime of GraphBLAS closeness centrality compared to NetworkX across all benchmark graphs. GraphBLAS achieves consistent speedups across all four graphs, ranging from approximately $4\times$ on `p2p-Gnutella25` to over $13\times$ on `ca-HepPh`.

Table 5: *Runtime comparison of closeness centrality between GraphBLAS and NetworkX*

Graph	GraphBLAS time	NetworkX time	Speedup
bcsstk13	0.677 s	6.344 s	9.371
ca-HepPh	15.567 s	209.137 s	13.434
p2p-Gnutella25	22.208 s	87.693 s	3.949
email-Enron	56.085 s	577.651 s	10.299

* Speedup is ratio of NetworkX time to GraphBLAS time

4. CONCLUSION

This work presented the implementation of two centrality metrics - Katz centrality and closeness centrality - in LAGraph. We motivated the need for scalable centrality algorithms by highlighting the limitations of traditional iterative, traversal-based approaches on large graphs, and demonstrated how the linear algebraic formulation of graph algorithms provides a mathematically concise and computationally efficient alternative. We discussed the theoretical foundations of both algorithms, described their implementations in SuiteSparse:GraphBLAS, and introduced `LAGraph_FW` as a supporting utility function for all-pairs shortest-path computation.

Our benchmarking results demonstrate that both implementations achieve considerable speedups over NetworkX across a range of real-world graphs. For Katz centrality, speedups range from $213\times$ to over $1{,}000\times$, with NetworkX failing to complete on larger graphs entirely. For closeness centrality, consistent speedups of $4\times$ to $13\times$ are observed. A comparison of the performance of Katz centrality against that of the GAP benchmark PageRank implementation confirms that the two algorithms share comparable per-iteration cost, with the performance gap attributable primarily to stricter convergence criteria and the use of `FP64` arithmetic rather than any fundamental inefficiency in the implementation. Overall, these results demonstrate that SuiteSparse:GraphBLAS provides an effective foundation for implementing complex graph algorithms concisely and efficiently.

4.1. Future Work

Several directions remain open for improving and extending this work. For Katz centrality, the convergence criterion warrants further investigation. The current implementation monitors the L1 norm of successive iterates directly, whereas NetworkX scales the tolerance by the number of nodes in the graph, a choice that weakens the stopping condition for large graphs and can lead to premature termination and inaccurate scores. A more principled convergence criterion, independent of graph size, should be explored and standardized. Additionally, the current implementation

operates exclusively in `FP64`. Adding `FP32` support would reduce memory traffic per iteration when possible and improve performance, bringing the implementation more in line with GAP benchmark PageRank.

For closeness centrality, several algorithmic improvements are worth pursuing. First, the current BFS-based implementation processes one source node at a time. A batched BFS approach, where multiple sources are processed simultaneously using matrix-matrix rather than matrix-vector operations, could significantly reduce total runtime, though this comes with increased memory usage that would need to be carefully managed. Second, the Floyd-Warshall implementation could be replaced or supplemented with an improved variant that reduces unnecessary computation on sparse graphs. Finally, the current implementation requires the caller to manually select a shortest-path algorithm. A more user-friendly interface would automatically select a suitable shortest-path subroutine based on graph properties cached in the LAGraph graph object, such as edge weights, symmetry structure, and graph size. While natural defaults exist, such as BFS for unweighted graphs or Bellman-Ford for graphs with negative weights, the optimal choice is not always straightforward and may depend on additional heuristics, making this a worthwhile area for future investigation.

REFERENCES

[1] G. Szárnyas et al., "LAGraph: Linear algebra, network analysis libraries, and the study of graph algorithms," in *2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2021, pp. 243–252. DOI: 10.1109/IPDPSW52791.2021.00046.

[2] T. A. Davis, "Algorithm 1000: Suitesparse:graphblas: Graph algorithms in the language of sparse linear algebra," *ACM Trans. Math. Softw.*, vol. 45, Dec. 2019.

[3] L. Katz, "A new status index derived from sociometric data analysis," *Psychometrika*, vol. 18, pp. 39–43, 1953.

[4] S. Beamer, K. Asanović, and D. Patterson, *The gap benchmark suite*, 2017. arXiv: 1508.03619 [cs.DC]. [Online]. Available: https://arxiv.org/abs/1508.03619.

[5] A. Bavelas, "Communication patterns in task-oriented groups," *Journal of the Acoustical Society of America*, vol. 22, no. 6, pp. 725–730, 1950. DOI: 10.1121/1.1906679.

[6] S. Wasserman and K. Faust, *Social Network Analysis: Methods and Applications*. Cambridge: Cambridge University Press, 1994.

[7] J. Leskovec and A. Krevl, *SNAP Datasets: Stanford large network dataset collection*, http://snap.stanford.edu/data, Jun. 2014.

[8] A. Azad et al., "Evaluation of graph analytics frameworks using the gap benchmark suite," in *2020 IEEE International Symposium on Workload Characterization (IISWC)*, 2020, pp. 216–227. DOI: 10.1109/IISWC50251.2020.00029.

APPENDIX A: KATZ CENTRALITY CODE

```
int LAGr_KatzCentrality
(
    // output:
    GrB_Vector *centrality,
    int64_t *iters,
    // input:
    LAGraph_Graph G,
    double alpha,
    double beta,
    int64_t max_iter,
    double tol,
    bool normalize,
    bool use_weights,  // uses weights if true, else treats all weights as 1
    char *msg
)
{
    //--------------------------------------------------------------------------
    // check inputs
    //--------------------------------------------------------------------------

    LG_CLEAR_MSG;
    GrB_Index n = 0;
    GrB_Vector x = NULL, x_prev = NULL, b = NULL, t = NULL;

    LG_ASSERT (centrality != NULL && iters != NULL, GrB_NULL_POINTER) ;
    (*centrality) = NULL;
    LG_TRY (LAGraph_CheckGraph (G, msg)) ;

    GrB_Matrix AT;
    if (G->kind == LAGraph_ADJACENCY_UNDIRECTED ||
        G->is_symmetric_structure == LAGraph_TRUE)
    {
        AT = G->A ;
    }
    else
    {
        AT = G->AT ;
        LG_ASSERT_MSG (AT != NULL, LAGRAPH_NOT_CACHED, "G->AT is required") ;
    }

    // compute correct semiring based on whether edge weights are used
    // TODO: add FP32 support
    GrB_Semiring semiring = use_weights ? GrB_PLUS_TIMES_SEMIRING_FP64
                                        : GxB_PLUS_SECOND_FP64 ;

    if (use_weights)
    {
        // ensure min edge weight is available and nonnegative
        LG_TRY (LAGraph_Cached_EMin (G, msg)) ;
```

```
    LG_ASSERT_MSG (
        G->emin != NULL &&
            (G->emin_state == LAGraph_VALUE ||
             G->emin_state == LAGraph_BOUND),
        LAGRAPH_NOT_CACHED, "G->emin is required") ;

    double emin = 0 ;
    GRB_TRY (GrB_Scalar_extractElement_FP64 (&emin, G->emin)) ;
    LG_ASSERT_MSG (emin >= 0, GrB_INVALID_VALUE,
        "use_weights=true requires nonnegative edge weights") ;
}

//------------------------------------------------------------------------------
// initializations
//------------------------------------------------------------------------------

GRB_TRY (GrB_Matrix_nrows (&n, AT)) ;

GRB_TRY (GrB_Vector_new (&x, GrB_FP64, n)) ;
GRB_TRY (GrB_Vector_new (&x_prev, GrB_FP64, n)) ;
GRB_TRY (GrB_Vector_new (&b, GrB_FP64, n)) ;
GRB_TRY (GrB_Vector_new (&t, GrB_FP64, n)) ;

GRB_TRY (GrB_assign (x, NULL, NULL, 0.0, GrB_ALL, n, NULL)) ;
GRB_TRY (GrB_assign (x_prev, NULL, NULL, 0.0, GrB_ALL, n, NULL)) ;
GRB_TRY (GrB_assign (b, NULL, NULL, beta, GrB_ALL, n, NULL)) ;

// first iteration is always done
double rdiff = 1 ;

// TODO: determine best way to stop
for ((*iters) = 0 ; ; (*iters)++)
{
    // check for convergence failure
    LG_ASSERT_MSGF ((*iters) < max_iter, LAGRAPH_CONVERGENCE_FAILURE,
        "katz centrality failed to converge in %" PRId64 " iterations",
         max_iter) ;

    // swap x and x_prev
    GrB_Vector temp = x_prev ;
    x_prev = x ;
    x = temp ;

    // x = A' * x_prev
    GRB_TRY (GrB_mxv (x, NULL, NULL, semiring, AT, x_prev, NULL)) ;

    // x = alpha * x + beta
    GRB_TRY (GrB_apply (x, NULL, NULL, GrB_TIMES_FP64, alpha, x, NULL)) ;
    GRB_TRY (GrB_eWiseAdd (x, NULL, NULL, GrB_PLUS_FP64, x, b, NULL)) ;

    // t = x - x_prev
    GRB_TRY (GrB_eWiseAdd (t, NULL, NULL, GrB_MINUS_FP64, x, x_prev,
        NULL)) ;
    // t = abs (t)
```

```
        GRB_TRY (GrB_apply (t, NULL, NULL, GrB_ABS_FP64, t, NULL)) ;
        // rdiff = sum (t)
        GRB_TRY (GrB_reduce (&rdiff, NULL, GrB_PLUS_MONOID_FP64, t, NULL)) ;

        if (rdiff < tol) break ;
    }

    // normalize using the L2 norm if flag is set
    if (normalize)
    {
        double sumsq = 0 ;

        // sumsq = x' * x = sum (x .* x)
        GRB_TRY (GrB_eWiseMult (t, NULL, NULL, GrB_TIMES_FP64, x, x, NULL)) ;
        GRB_TRY (GrB_reduce (&sumsq, NULL, GrB_PLUS_MONOID_FP64, t, NULL)) ;

        if (sumsq > 0)
        {
            GRB_TRY (GrB_apply (x, NULL, NULL, GrB_DIV_FP64, x, sqrt (sumsq),
                NULL)) ;
        }
    }

    //--------------------------------------------------------------------------
    // free workspace and return result
    //--------------------------------------------------------------------------

    (*centrality) = x;
    LG_FREE_WORK;

    return GrB_SUCCESS;
}
```

APPENDIX B: CLOSENESS CENTRALITY CODE

```
int LAGr_ClosenessCentrality
(
    // output:
    GrB_Vector *centrality,
    // input:
    LAGraph_Graph G,
    GrB_Vector sources,     // nodes to score; NULL or empty => all nodes
    bool use_weights,       // if true, use edge weights in shortest paths
    cc_algo_t algorithm,    // shortest-path algorithm to use
    GrB_Scalar Delta,       // delta for SSSP; if NULL, derived from G->emin
    char *msg
)
{

    //--------------------------------------------------------------------------
    // check inputs
    //--------------------------------------------------------------------------

    LG_CLEAR_MSG;
    GrB_Info info;
    GrB_Vector centrality_vector = NULL;
    GrB_Vector distance_vector = NULL;  // per-node BF / SSSP result
    GrB_Vector bfs_level = NULL;        // BFS: level (distance) of each node
    GrB_Scalar inf_scalar = NULL;       // INFINITY threshold for SSSP pruning
    GrB_Vector dist_sums = NULL;        // FW: row sums of D
    GrB_Vector reachable_counts = NULL; // FW: row non-zero counts of D
    GrB_Matrix D_APSP = NULL;           // FW: APSP result
    GrB_Matrix A_work = NULL;           // AT copy for G_AT
    LAGraph_Graph G_AT = NULL;          // temporary graph wrapping AT
                                         // (directed only)
    GrB_Vector x = NULL;                // FW: dense all-zeros vector for
                                         // row counting
    GrB_Index *source_indices = NULL;
    GrB_Index n = 0;
    GrB_Index source_count = 0;

    LG_ASSERT(centrality != NULL, GrB_NULL_POINTER);
    (*centrality) = NULL;
    LG_TRY(LAGraph_CheckGraph(G, msg));

    GRB_TRY(GrB_Matrix_nrows(&n, G->A));

    bool use_all_nodes = (sources == NULL);
    if (!use_all_nodes)
    {
        GRB_TRY(GrB_Vector_nvals(&source_count, sources));
        use_all_nodes = (source_count == 0);
    }
```

```
cc_algo_t algo = algorithm;

// set up G_AT: graph over the incoming adjacency. Running any
// shortest-path algorithm from v on G_AT traverses incoming edges,
// giving distances to v in G.

bool is_directed = (G->kind != LAGraph_ADJACENCY_UNDIRECTED &&
                    G->is_symmetric_structure != LAGraph_TRUE);

if (is_directed)
{
    LG_ASSERT_MSG(G->AT != NULL, LAGRAPH_NOT_CACHED, "G->AT is required");
    GRB_TRY(GrB_Matrix_dup(&A_work, G->AT));
    LG_TRY(LAGraph_New(&G_AT, &A_work,
                       LAGraph_ADJACENCY_DIRECTED, msg));

    if (G->emin != NULL)
    {
        GRB_TRY(GrB_Scalar_dup(&G_AT->emin, G->emin));
        G_AT->emin_state = G->emin_state;
    }
}

// G_trav->A is the incoming adjacency used by all algorithm paths.
LAGraph_Graph G_trav = (G_AT != NULL) ? G_AT : G;

//------------------------------------------------------------------------
// build source node list
//------------------------------------------------------------------------

if (!use_all_nodes)
{
    LG_TRY(LAGraph_Malloc((void **)&source_indices,
                          source_count, sizeof(GrB_Index), msg));
    GRB_TRY(GrB_Vector_extractTuples_UINT64 (source_indices, NULL,
                                    &source_count, sources));
    for (GrB_Index k = 0; k < source_count; k++)
    {
        LG_ASSERT(source_indices[k] < n, GrB_INVALID_INDEX);
    }
}
else
{
    source_count = n;
}

//------------------------------------------------------------------------
// allocate output vector
//------------------------------------------------------------------------

GRB_TRY(GrB_Vector_new(&centrality_vector, GrB_FP64, n));
if (use_all_nodes)
{
    // Dense output: isolated nodes keep score 0.
```

```
    GRB_TRY(GrB_assign(centrality_vector, NULL, NULL,
                       0.0, GrB_ALL, n, NULL));
}

//============================================================================
// Floyd-Warshall path
//============================================================================

if (algo == CC_FLOYD_WARSHALL)
{
    GrB_Type D_type;
    GRB_TRY(LAGraph_FW(G_trav->A, &D_APSP, &D_type));

    // Remove self-loops so they do not bias sums or counts.
    GRB_TRY(GrB_select(D_APSP, NULL, NULL,
                       GrB_OFFDIAG, D_APSP, (int64_t)0, NULL));

    // dist_sums[v] = sum of row v
    GRB_TRY(GrB_Vector_new(&dist_sums, GrB_FP64, n));
    GRB_TRY(GrB_reduce(dist_sums, NULL, NULL,
                       GrB_PLUS_MONOID_FP64, D_APSP, NULL));

    // reachable_counts[v] = number of non-zeros in row v of D_APSP.
    // Multiply D_APSP by a dense all-zeros vector with
    // LAGraph_plus_one_fp64:
    GRB_TRY(GrB_Vector_new(&x, GrB_FP64, n));
    GRB_TRY(GrB_assign(x, NULL, NULL, (double)0, GrB_ALL, n, NULL));
    GRB_TRY(GrB_Vector_new(&reachable_counts, GrB_FP64, n));
    GRB_TRY(GrB_mxv(reachable_counts, NULL, NULL, LAGraph_plus_one_fp64,
            D_APSP, x, NULL));
    GRB_TRY(GrB_free(&D_APSP));

    // centrality[v] = R(v) / dist_sums[v].
    GRB_TRY(GrB_eWiseMult(centrality_vector, NULL, NULL, GrB_DIV_FP64,
                  reachable_counts, dist_sums, NULL));

    (*centrality) = centrality_vector;
    LG_FREE_WORK;
    return GrB_SUCCESS;
}

if (algo == CC_SSSP)
{
    GRB_TRY(GrB_Scalar_new(&inf_scalar, GrB_FP64));
    GRB_TRY(GrB_Scalar_setElement_FP64(inf_scalar, (double)INFINITY));
}

//============================================================================
// per-node shortest-path loop
//============================================================================

for (GrB_Index k = 0; k < source_count; k++)
{
    GrB_Index node_to_score =
```

```
    use_all_nodes ? k : source_indices[k];

double distance_sum = 0;
GrB_Index reachable_count = 0;

//------------------------------------------------------------------
// BFS (unweighted shortest paths)
//------------------------------------------------------------------

if (algo == CC_BFS)
{
    // IMPORTANT: LAGr_BreadthFirstSearch sets *level = NULL without
    // freeing the prior handle.  We must free bfs_level before every
    // call to avoid a memory leak.
    GRB_TRY(GrB_free(&bfs_level));
    LG_TRY(LAGr_BreadthFirstSearch(&bfs_level, NULL,
                                   G_trav, node_to_score, msg));

    GRB_TRY(GrB_Vector_removeElement(bfs_level, node_to_score));
    GRB_TRY(GrB_Vector_nvals(&reachable_count, bfs_level));
    if (reachable_count > 0)
    {
        GRB_TRY(GrB_reduce(&distance_sum, NULL,
                           GrB_PLUS_MONOID_FP64, bfs_level, NULL));
    }
}

//------------------------------------------------------------------
// Delta-stepping SSSP (Dijkstra-like, non-negative weights)
//------------------------------------------------------------------

else if (algo == CC_SSSP)
{
    GRB_TRY(GrB_free(&distance_vector));
    LG_TRY(LAGr_SingleSourceShortestPath(&distance_vector,
                                         G_trav,
                                         node_to_score,
                                         Delta, msg));

    // Prune unreachable nodes (value == INFINITY).
    GRB_TRY(GrB_select(distance_vector, NULL, NULL,
                       GrB_VALUELT_FP64,
                       distance_vector, inf_scalar, NULL));

    // Exclude self (distance = 0).
    GRB_TRY(GrB_Vector_removeElement(distance_vector,
                                     node_to_score));
    GRB_TRY(GrB_Vector_nvals(&reachable_count, distance_vector));

    if (reachable_count > 0)
    {
        GRB_TRY(GrB_reduce(&distance_sum, NULL,
                           GrB_PLUS_MONOID_FP64,
                           distance_vector, NULL));
```

```
            }
        }

        //----------------------------------------------------------------------
        // Bellman-Ford
        //----------------------------------------------------------------------

        else // CC_BELLMAN_FORD
        {
            GRB_TRY(GrB_free(&distance_vector));

            info = LAGraph_BF_basic(&distance_vector,
                                    G_trav->A, node_to_score);
            if (info == GrB_NO_VALUE)
            {
                // Negative-weight cycle reachable from this node;
                // For now, set centrality to NaN to indicate an invalid
                // score, and continue.
                GRB_TRY(GrB_Vector_setElement(centrality_vector,
                                              (double)NAN, node_to_score));
                continue;
            }
            GRB_TRY(info);

            // Exclude self (distance = 0).
            GRB_TRY(GrB_Vector_removeElement(distance_vector,
                                             node_to_score));
            GRB_TRY(GrB_Vector_nvals(&reachable_count, distance_vector));

            if (reachable_count > 0)
            {
                GRB_TRY(GrB_reduce(&distance_sum, NULL,
                                   GrB_PLUS_MONOID_FP64,
                                   distance_vector, NULL));
            }
        }

        // closeness(v) = R(v) / sum_{u->v} d(u, v)
        if (reachable_count > 0 && distance_sum > 0)
        {
            double closeness = ((double)reachable_count) / distance_sum;
            GRB_TRY(GrB_Vector_setElement(centrality_vector,
                                          closeness, node_to_score));
        }
    }

    //----------------------------------------------------------------------
    // free workspace and return result
    //----------------------------------------------------------------------

    (*centrality) = centrality_vector;
    LG_FREE_WORK;
    return GrB_SUCCESS;
}
```